

```

import React, { useState, useEffect } from 'react';
import { Truck, Clock, DollarSign, MapPin, Cloud, Menu, Navigation, Fuel, Coffee, Scale,
Package, Wrench, Calendar, FileText, MessageSquare, CheckCircle, AlertTriangle, Plus, X,
Send, Download } from 'lucide-react';

export default function TruckerApp() {
  const [activeTab, setActiveTab] = useState('dashboard');
  const [showInstallPrompt, setShowInstallPrompt] = useState(false);
  const [deferredPrompt, setDeferredPrompt] = useState(null);

  // Load data from storage on mount
  const [hosLog, setHosLog] = useState(() => {
    const saved = localStorage.getItem('hosLog');
    return saved ? JSON.parse(saved) : {
      driving: 0,
      onDuty: 0,
      offDuty: 8,
      sleeper: 0
    };
  });

  const [expenses, setExpenses] = useState(() => {
    const saved = localStorage.getItem('expenses');
    return saved ? JSON.parse(saved) : [
      { id: 1, type: 'Fuel', amount: 245.50, location: 'Atlanta, GA', date: '2025-11-24' },
      { id: 2, type: 'Maintenance', amount: 125.00, location: 'Nashville, TN', date: '2025-11-23' }
    ];
  });

  const [loads, setLoads] = useState(() => {
    const saved = localStorage.getItem('loads');
    return saved ? JSON.parse(saved) : [
      {
        id: 1,
        pickup: 'Atlanta, GA',
        delivery: 'Charlotte, NC',
        weight: '42,000 lbs',
        rate: '$1,200',
        pickupDate: '2025-11-26',
        deliveryDate: '2025-11-27',
        status: 'In Transit',
        cargo: 'Electronics'
      },
      {

```

```

      id: 2,
      pickup: 'Charlotte, NC',
      delivery: 'Richmond, VA',
      weight: '38,500 lbs',
      rate: '$950',
      pickupDate: '2025-11-27',
      deliveryDate: '2025-11-28',
      status: 'Scheduled',
      cargo: 'Auto Parts'
    }
  ];
});

```

```

const [maintenance, setMaintenance] = useState(() => {
  const saved = localStorage.getItem('maintenance');
  return saved ? JSON.parse(saved) : [
    { id: 1, item: 'Oil Change', dueDate: '2025-12-05', miles: 5000, status: 'upcoming' },
    { id: 2, item: 'Tire Rotation', dueDate: '2025-12-15', miles: 8000, status: 'upcoming' },
    { id: 3, item: 'DOT Inspection', dueDate: '2025-11-30', miles: 2000, status: 'urgent' }
  ];
});

```

```

const [messages, setMessages] = useState(() => {
  const saved = localStorage.getItem('messages');
  return saved ? JSON.parse(saved) : [
    { id: 1, from: 'Dispatch', text: 'Your next load is ready for pickup at 0800 tomorrow', time: '2h ago', unread: true },
    { id: 2, from: 'Fleet Manager', text: 'Please submit your fuel receipts from this week', time: '5h ago', unread: false },
    { id: 3, from: 'Support', text: 'Weather advisory: Heavy rain expected on I-85', time: '1d ago', unread: false }
  ];
});

```

// Save to storage whenever data changes

```

useEffect(() => {
  localStorage.setItem('hosLog', JSON.stringify(hosLog));
}, [hosLog]);

```

```

useEffect(() => {
  localStorage.setItem('expenses', JSON.stringify(expenses));
}, [expenses]);

```

```

useEffect(() => {

```

```
    localStorage.setItem('loads', JSON.stringify(loads));
  }, [loads]);
```

```
useEffect(() => {
  localStorage.setItem('maintenance', JSON.stringify(maintenance));
}, [maintenance]);
```

```
useEffect(() => {
  localStorage.setItem('messages', JSON.stringify(messages));
}, [messages]);
```

```
// PWA Install prompt
useEffect(() => {
  const handler = (e) => {
    e.preventDefault();
    setDeferredPrompt(e);
    setShowInstallPrompt(true);
  };

```

```
  window.addEventListener('beforeinstallprompt', handler);
```

```
  return () => window.removeEventListener('beforeinstallprompt', handler);
}, []);
```

```
const handleInstallClick = async () => {
  if (!deferredPrompt) return;

```

```
  deferredPrompt.prompt();
  const { outcome } = await deferredPrompt.userChoice;

```

```
  if (outcome === 'accepted') {
    setShowInstallPrompt(false);
  }

```

```
  setDeferredPrompt(null);
};
```

```
const [newExpense, setNewExpense] = useState({ type: 'Fuel', amount: "", location: "" });
const [showAddLoad, setShowAddLoad] = useState(false);
const [newLoad, setNewLoad] = useState({
  pickup: "",
  delivery: "",
  weight: "",
  rate: "",

```

```

    pickupDate: "",
    deliveryDate: "",
    cargo: ""
  });
  const [newMessage, setNewMessage] = useState("");

  const [trips, setTrips] = useState([
    {
      id: 1,
      name: 'Atlanta to NYC Run',
      origin: 'Atlanta, GA',
      destination: 'New York, NY',
      stops: ['Charlotte, NC', 'Richmond, VA', 'Philadelphia, PA'],
      distance: '875 miles',
      duration: '13h 30m',
      startDate: '2025-11-26'
    }
  ]);

  const [documents, setDocuments] = useState([
    { id: 1, name: 'CDL License', category: 'License', uploadDate: '2025-01-15', expires: '2027-01-15' },
    { id: 2, name: 'Medical Card', category: 'Medical', uploadDate: '2025-06-10', expires: '2026-06-10' },
    { id: 3, name: 'Insurance Certificate', category: 'Insurance', uploadDate: '2025-03-20', expires: '2026-03-20' },
    { id: 4, name: 'Bill of Lading - Load #1234', category: 'Load Documents', uploadDate: '2025-11-24', expires: null }
  ]);

  const addExpense = () => {
    if (newExpense.amount && newExpense.location) {
      setExpenses([
        {
          id: Date.now(),
          type: newExpense.type,
          amount: parseFloat(newExpense.amount),
          location: newExpense.location,
          date: new Date().toLocaleString().split('T')[0]
        },
        ...expenses
      ]);
      setNewExpense({ type: 'Fuel', amount: "", location: "" });
    }
  }

```

```
};
```

```
const addLoad = () => {  
  if (newLoad.pickup && newLoad.delivery) {  
    setLoads([  
      {  
        id: Date.now(),  
        ...newLoad,  
        status: 'Scheduled'  
      },  
      ...loads  
    ]);  
    setNewLoad({  
      pickup: "",  
      delivery: "",  
      weight: "",  
      rate: "",  
      pickupDate: "",  
      deliveryDate: "",  
      cargo: ""  
    });  
    setShowAddLoad(false);  
  }  
};
```

```
const updateLoadStatus = (id, status) => {  
  setLoads(loads.map(load => load.id === id ? { ...load, status } : load));  
};
```

```
const updateHOS = (category, value) => {  
  const total = Object.values(hosLog).reduce((a, b) => a + b, 0);  
  if (total - hosLog[category] + value <= 24) {  
    setHosLog({ ...hosLog, [category]: value });  
  }  
};
```

```
const nearbyAmenities = [  
  { name: 'Pilot Travel Center', type: 'Fuel', distance: '2.3 mi', icon: Fuel },  
  { name: "Love's Travel Stop", type: 'Food & Fuel', distance: '5.1 mi', icon: Coffee },  
  { name: 'CAT Scale', type: 'Weigh Station', distance: '3.7 mi', icon: Scale },  
  { name: 'Rest Area - I-85', type: 'Rest Stop', distance: '8.2 mi', icon: Coffee }  
];
```

```
const renderDashboard = () => (
```

```

<div className="space-y-4">
  {showInstallPrompt && (
    <div className="bg-gradient-to-r from-green-600 to-green-700 text-white p-4 rounded-lg shadow-lg">
      <div className="flex items-center justify-between">
        <div className="flex items-center flex-1">
          <Download className="w-8 h-8 mr-3" />
          <div>
            <p className="font-bold text-lg">Install TruckMate</p>
            <p className="text-sm text-green-100">Add to home screen for quick access</p>
          </div>
        </div>
        <button
          onClick={handleInstallClick}
          className="bg-white text-green-700 px-4 py-2 rounded-lg font-bold hover:bg-green-50 transition ml-2"
        >
          Install
        </button>
      </div>
    </div>
  )}

```

```

<div className="bg-gradient-to-r from-blue-600 to-blue-800 text-white p-6 rounded-lg shadow-lg">
  <div className="flex items-center justify-between">
    <div>
      <h2 className="text-2xl font-bold">Welcome Back, Driver</h2>
      <p className="text-blue-100 mt-1">Current Location: Alpharetta, GA</p>
    </div>
    <Truck className="w-16 h-16 opacity-80" />
  </div>
</div>

```

```

<div className="grid grid-cols-2 gap-4">
  <div className="bg-white p-4 rounded-lg shadow border-l-4 border-green-500">
    <div className="flex items-center justify-between">
      <div>
        <p className="text-gray-600 text-sm">Driving Time Left</p>
        <p className="text-2xl font-bold text-gray-900">{11 - hosLog.driving}h</p>
      </div>
      <Clock className="w-10 h-10 text-green-500" />
    </div>
  </div>
</div>

```

```

<div className="bg-white p-4 rounded-lg shadow border-l-4 border-orange-500">
  <div className="flex items-center justify-between">
    <div>
      <p className="text-gray-600 text-sm">Active Loads</p>
      <p className="text-2xl font-bold text-gray-900">{loads.filter(l => l.status === 'In
Transit').length}</p>
    </div>
    <Package className="w-10 h-10 text-orange-500" />
  </div>
</div>
</div>

{maintenance.filter(m => m.status === 'urgent').length > 0 && (
  <div className="bg-white p-4 rounded-lg shadow">
    <h3 className="font-bold text-gray-900 mb-3 flex items-center">
      <AlertTriangle className="w-5 h-5 mr-2 text-red-600" />
      Urgent Maintenance
    </h3>
    {maintenance.filter(m => m.status === 'urgent').map(item => (
      <div key={item.id} className="bg-red-50 border-l-4 border-red-400 p-3 rounded mb-2">
        <p className="text-sm font-medium text-red-800">{item.item}</p>
        <p className="text-xs text-red-600">Due: {item.dueDate} ({item.miles} miles)</p>
      </div>
    ))}
  </div>
)}

<div className="bg-white p-4 rounded-lg shadow">
  <h3 className="font-bold text-gray-900 mb-3 flex items-center">
    <Package className="w-5 h-5 mr-2 text-blue-600" />
    Current Load
  </h3>
  {loads.filter(l => l.status === 'In Transit').slice(0, 1).map(load => (
    <div key={load.id} className="bg-blue-50 p-4 rounded-lg">
      <div className="flex justify-between items-start mb-2">
        <div>
          <p className="font-medium text-gray-900">{load.cargo}</p>
          <p className="text-sm text-gray-600">{load.pickup} → {load.delivery}</p>
        </div>
        <span className="px-2 py-1 bg-blue-600 text-white text-xs
rounded-full">{load.status}</span>
      </div>
      <div className="grid grid-cols-2 gap-2 mt-3 text-sm">

```

```

    <div>
      <p className="text-gray-600">Weight</p>
      <p className="font-medium">{load.weight}</p>
    </div>
    <div>
      <p className="text-gray-600">Rate</p>
      <p className="font-medium">{load.rate}</p>
    </div>
  </div>
  </div>
  </div>
  )))
  {loads.filter(l => l.status === 'In Transit').length === 0 && (
    <p className="text-gray-500 text-center py-4">No active loads</p>
  )}
</div>

<div className="bg-white p-4 rounded-lg shadow">
  <h3 className="font-bold text-gray-900 mb-3 flex items-center">
    <MapPin className="w-5 h-5 mr-2 text-red-600" />
    Nearby Amenities
  </h3>
  <div className="space-y-2">
    {nearbyAmenities.slice(0, 3).map((amenity, idx) => (
      <div key={idx} className="flex items-center justify-between p-3 bg-gray-50 rounded
hover:bg-gray-100 transition">
        <div className="flex items-center">
          <amenity.icon className="w-5 h-5 text-gray-600 mr-3" />
          <div>
            <p className="font-medium text-gray-900">{amenity.name}</p>
            <p className="text-sm text-gray-600">{amenity.type}</p>
          </div>
        </div>
        <p className="text-sm font-medium text-blue-600">{amenity.distance}</p>
      </div>
    ))}
  </div>
</div>
</div>
);

const renderLoads = () => (
  <div className="space-y-4">
    <div className="flex justify-between items-center">
      <h2 className="text-2xl font-bold text-gray-900">Loads & Deliveries</h2>

```

```

<button
  onClick={() => setShowAddLoad(true)}
  className="bg-blue-600 text-white p-2 rounded-lg hover:bg-blue-700 transition"
>
  <Plus className="w-6 h-6" />
</button>
</div>

{showAddLoad && (
  <div className="bg-white p-6 rounded-lg shadow-lg border-2 border-blue-500">
    <div className="flex justify-between items-center mb-4">
      <h3 className="font-bold text-gray-900 text-xl">Add New Load</h3>
      <button onClick={() => setShowAddLoad(false)} className="text-gray-500
hover:text-gray-700">
        <X className="w-6 h-6" />
      </button>
    </div>

    <div className="space-y-3">
      <div>
        <label className="block text-sm font-medium text-gray-700 mb-1">Pickup
Location</label>
        <input
          type="text"
          value={newLoad.pickup}
          onChange={(e) => setNewLoad({ ...newLoad, pickup: e.target.value })}
          placeholder="City, State"
          className="w-full px-3 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500"
        />
      </div>

      <div>
        <label className="block text-sm font-medium text-gray-700 mb-1">Delivery
Location</label>
        <input
          type="text"
          value={newLoad.delivery}
          onChange={(e) => setNewLoad({ ...newLoad, delivery: e.target.value })}
          placeholder="City, State"
          className="w-full px-3 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500"
        />
      </div>
    </div>
  </div>
)

```

```

<div className="grid grid-cols-2 gap-3">
  <div>
    <label className="block text-sm font-medium text-gray-700 mb-1">Weight</label>
    <input
      type="text"
      value={newLoad.weight}
      onChange={(e) => setNewLoad({ ...newLoad, weight: e.target.value })}
      placeholder="40,000 lbs"
      className="w-full px-3 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500"
    />
  </div>

  <div>
    <label className="block text-sm font-medium text-gray-700 mb-1">Rate</label>
    <input
      type="text"
      value={newLoad.rate}
      onChange={(e) => setNewLoad({ ...newLoad, rate: e.target.value })}
      placeholder="$1,000"
      className="w-full px-3 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500"
    />
  </div>
</div>

<div>
  <label className="block text-sm font-medium text-gray-700 mb-1">Cargo</label>
  <input
    type="text"
    value={newLoad.cargo}
    onChange={(e) => setNewLoad({ ...newLoad, cargo: e.target.value })}
    placeholder="Electronics, Auto Parts, etc."
    className="w-full px-3 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500"
  />
</div>

<div className="grid grid-cols-2 gap-3">
  <div>
    <label className="block text-sm font-medium text-gray-700 mb-1">Pickup
Date</label>
    <input

```

```

      type="date"
      value={newLoad.pickupDate}
      onChange={(e) => setNewLoad({ ...newLoad, pickupDate: e.target.value })}
      className="w-full px-3 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500"
    />
  </div>

```

```

  <div>
    <label className="block text-sm font-medium text-gray-700 mb-1">Delivery
Date</label>
    <input
      type="date"
      value={newLoad.deliveryDate}
      onChange={(e) => setNewLoad({ ...newLoad, deliveryDate: e.target.value })}
      className="w-full px-3 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500"
    />
  </div>
</div>

```

```

  <button
    onClick={addLoad}
    className="w-full bg-blue-600 text-white py-3 rounded-lg font-medium
hover:bg-blue-700 transition"
  >
    Add Load
  </button>
</div>
</div>
)}

```

```

<div className="space-y-3">
  {loads.map((load) => (
    <div key={load.id} className="bg-white p-4 rounded-lg shadow">
      <div className="flex justify-between items-start mb-3">
        <div>
          <p className="font-bold text-gray-900">{load.cargo}</p>
          <p className="text-sm text-gray-600 mt-1">{load.pickup} → {load.delivery}</p>
        </div>
        <span className={`px-3 py-1 rounded-full text-xs font-medium ${
          load.status === 'In Transit' ? 'bg-green-100 text-green-800' :
          load.status === 'Delivered' ? 'bg-blue-100 text-blue-800' :
          'bg-yellow-100 text-yellow-800'
        }`}

```

```

    }}>
    {load.status}
  </span>
</div>

<div className="grid grid-cols-2 gap-3 mb-3 text-sm">
  <div>
    <p className="text-gray-600">Weight</p>
    <p className="font-medium text-gray-900">{load.weight}</p>
  </div>
  <div>
    <p className="text-gray-600">Rate</p>
    <p className="font-medium text-gray-900">{load.rate}</p>
  </div>
  <div>
    <p className="text-gray-600">Pickup</p>
    <p className="font-medium text-gray-900">{load.pickupDate}</p>
  </div>
  <div>
    <p className="text-gray-600">Delivery</p>
    <p className="font-medium text-gray-900">{load.deliveryDate}</p>
  </div>
</div>

{load.status !== 'Delivered' && (
  <div className="flex gap-2">
    {load.status === 'Scheduled' && (
      <button
        onClick={() => updateLoadStatus(load.id, 'In Transit')}
        className="flex-1 bg-green-600 text-white py-2 rounded-lg text-sm font-medium
hover:bg-green-700 transition"
      >
        Start Transit
      </button>
    )}
    {load.status === 'In Transit' && (
      <button
        onClick={() => updateLoadStatus(load.id, 'Delivered')}
        className="flex-1 bg-blue-600 text-white py-2 rounded-lg text-sm font-medium
hover:bg-blue-700 transition"
      >
        Mark Delivered
      </button>
    )}
  </div>
)}

```

```

    </div>
  })
</div>
  )))
</div>
</div>
);

```

```

const renderMaintenance = () => (
  <div className="space-y-4">
    <h2 className="text-2xl font-bold text-gray-900">Maintenance Reminders</h2>

    {maintenance.filter(m => m.status === 'urgent').length > 0 && (
      <div className="bg-white p-4 rounded-lg shadow">
        <h3 className="font-bold text-red-600 mb-3 flex items-center">
          <AlertTriangle className="w-5 h-5 mr-2" />
          Urgent Items
        </h3>
        {maintenance.filter(m => m.status === 'urgent').map(item => (
          <div key={item.id} className="bg-red-50 border-l-4 border-red-500 p-4 rounded mb-3">
            <div className="flex justify-between items-start">
              <div>
                <p className="font-bold text-gray-900">{item.item}</p>
                <p className="text-sm text-gray-600 mt-1">Due: {item.dueDate}</p>
                <p className="text-sm text-gray-600">Remaining: {item.miles} miles</p>
              </div>
              <CheckCircle className="w-6 h-6 text-red-500" />
            </div>
          </div>
        ))}
      </div>
    )}

    <div className="bg-white p-4 rounded-lg shadow">
      <h3 className="font-bold text-yellow-600 mb-3 flex items-center">
        <Clock className="w-5 h-5 mr-2" />
        Upcoming Items
      </h3>
      {maintenance.filter(m => m.status === 'upcoming').map(item => (
        <div key={item.id} className="bg-yellow-50 border-l-4 border-yellow-500 p-4 rounded
mb-3">
          <div className="flex justify-between items-start">
            <div>
              <p className="font-bold text-gray-900">{item.item}</p>

```

```

      <p className="text-sm text-gray-600 mt-1">Due: {item.dueDate}</p>
      <p className="text-sm text-gray-600">Remaining: {item.miles} miles</p>
    </div>
    <CheckCircle className="w-6 h-6 text-yellow-500" />
  </div>
</div>
  )}
</div>

```

```

    <button className="w-full bg-blue-600 text-white py-3 rounded-lg font-medium
    hover:bg-blue-700 transition flex items-center justify-center">
      <Plus className="w-5 h-5 mr-2" />
      Add Maintenance Item
    </button>
  </div>
);

```

```

const renderTrips = () => (
  <div className="space-y-4">
    <h2 className="text-2xl font-bold text-gray-900">Trip Planning</h2>

    {trips.map(trip => (
      <div key={trip.id} className="bg-white p-5 rounded-lg shadow">
        <div className="flex justify-between items-start mb-3">
          <h3 className="font-bold text-gray-900 text-lg">{trip.name}</h3>
          <Navigation className="w-6 h-6 text-blue-600" />
        </div>

        <div className="space-y-3">
          <div>
            <p className="text-sm text-gray-600">Route</p>
            <p className="font-medium text-gray-900">{trip.origin} → {trip.destination}</p>
          </div>

          <div>
            <p className="text-sm text-gray-600 mb-1">Stops</p>
            {trip.stops.map((stop, idx) => (
              <div key={idx} className="flex items-center text-sm text-gray-700 mb-1">
                <MapPin className="w-4 h-4 mr-2 text-blue-600" />
                {stop}
              </div>
            ))}
          </div>
        </div>
      </div>
    ))}
  </div>

```

```

<div className="grid grid-cols-3 gap-3 pt-3 border-t">
  <div>
    <p className="text-xs text-gray-600">Distance</p>
    <p className="font-medium text-gray-900">{trip.distance}</p>
  </div>
  <div>
    <p className="text-xs text-gray-600">Duration</p>
    <p className="font-medium text-gray-900">{trip.duration}</p>
  </div>
  <div>
    <p className="text-xs text-gray-600">Start Date</p>
    <p className="font-medium text-gray-900">{trip.startDate}</p>
  </div>
</div>

<button className="w-full mt-4 bg-blue-600 text-white py-2 rounded-lg font-medium
hover:bg-blue-700 transition">
  View Route Details
</button>
</div>
)}}

<button className="w-full bg-green-600 text-white py-3 rounded-lg font-medium
hover:bg-green-700 transition flex items-center justify-center">
  <Plus className="w-5 h-5 mr-2" />
  Plan New Trip
</button>
</div>
);

const renderDocuments = () => (
  <div className="space-y-4">
    <h2 className="text-2xl font-bold text-gray-900">Document Storage</h2>

    <div className="bg-white p-4 rounded-lg shadow">
      <div className="grid grid-cols-2 gap-3 mb-4">
        <button className="bg-blue-600 text-white py-2 rounded-lg text-sm font-medium
hover:bg-blue-700 transition">
          Upload Document
        </button>
        <button className="bg-gray-200 text-gray-700 py-2 rounded-lg text-sm font-medium
hover:bg-gray-300 transition">
          Scan Document

```

```

        </button>
      </div>
    </div>

    {[ 'License', 'Medical', 'Insurance', 'Load Documents' ].map(category => (
      <div key={category} className="bg-white p-4 rounded-lg shadow">
        <h3 className="font-bold text-gray-900 mb-3">{category}</h3>
        {documents.filter(doc => doc.category === category).map(doc => (
          <div key={doc.id} className="flex items-center justify-between p-3 bg-gray-50 rounded
mb-2">
            <div className="flex items-center">
              <FileText className="w-5 h-5 text-blue-600 mr-3" />
              <div>
                <p className="font-medium text-gray-900 text-sm">{doc.name}</p>
                <p className="text-xs text-gray-600">Uploaded: {doc.uploadDate}</p>
                {doc.expires && (
                  <p className="text-xs text-orange-600">Expires: {doc.expires}</p>
                )}
              </div>
            </div>
            <button className="text-blue-600 text-sm font-medium">View</button>
          </div>
        )]}
      </div>
    )]}
  </div>
)}
</div>
);

```

```

const renderMessages = () => (
  <div className="space-y-4">
    <h2 className="text-2xl font-bold text-gray-900">Messages</h2>

    <div className="bg-white rounded-lg shadow">
      {messages.map(msg => (
        <div key={msg.id} className={`p-4 border-b border-gray-200 ${msg.unread ?
'bg-blue-50' : ''}`}>
          <div className="flex justify-between items-start mb-2">
            <p className="font-bold text-gray-900">{msg.from}</p>
            <p className="text

```